Report on Movie Rating Predictions in R

Isaac Sinaga

2025-08-10

Introduction

Entertainment platforms often use built-in models that try to predict user preferences by filtering out content. Platforms such as Netflix, Spotify, and YouTube use these types of systems to ensure that the user enjoys the best content.

In this report, we will create a model that replicates this type of system and predict different movie ratings that would be given by users. The dataset that we will be using is called the Movielens 10M dataset, given to us by GroupLens. Our goal for creating this prediction model is to achieve a Random Mean Squared Error (RMSE) below 0.86490.

Preparing the Data

We will download the 10M Movielens dataset given to us. The dataset contains approximately 10M different movie ratings. Next, we will split the dataset, setting aside 90% for the training set, which we will call 'edx', and 10% for the testing set, which we will call 'final_holdout_test'. We will use the 'edx' training set to perform some data analysis and build our model and make our predictions, and at the end we will use our 'final_holdout_test' set to validate our model.

```r
##########################################################
# Create edx and final_holdout_test sets
##########################################################

# Note: this process could take a couple of minutes

if(!require(tidyverse)) install.packages("tidyverse", repos = "http://cran.us.r-project.org")
```

```
## Loading required package: tidyverse

## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.2     v tibble    3.2.1
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.0.4
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
if(!require(caret)) install.packages("caret", repos = "http://cran.us.r-project.org")
```

```
## Loading required package: caret
## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
##     lift
```

```r
library(tidyverse)
library(caret)

# MovieLens 10M dataset:
# https://grouplens.org/datasets/movielens/10m/
# http://files.grouplens.org/datasets/movielens/ml-10m.zip

options(timeout = 120)

dl <- "ml-10M100K.zip"
if(!file.exists(dl))
  download.file("https://files.grouplens.org/datasets/movielens/ml-10m.zip", dl)

ratings_file <- "ml-10M100K/ratings.dat"
if(!file.exists(ratings_file))
  unzip(dl, ratings_file)

movies_file <- "ml-10M100K/movies.dat"
if(!file.exists(movies_file))
  unzip(dl, movies_file)

ratings <- as.data.frame(str_split(read_lines(ratings_file), fixed("::"), simplify = TRUE),
                         stringsAsFactors = FALSE)
colnames(ratings) <- c("userId", "movieId", "rating", "timestamp")
ratings <- ratings %>%
  mutate(userId = as.integer(userId),
         movieId = as.integer(movieId),
         rating = as.numeric(rating),
         timestamp = as.integer(timestamp))

movies <- as.data.frame(str_split(read_lines(movies_file), fixed("::"), simplify = TRUE),
                        stringsAsFactors = FALSE)
colnames(movies) <- c("movieId", "title", "genres")
movies <- movies %>%
  mutate(movieId = as.integer(movieId))

movielens <- left_join(ratings, movies, by = "movieId")

# Final hold-out test set will be 10% of MovieLens data
set.seed(1) # if using R 3.6 or later
# set.seed(1) # if using R 3.5 or earlier
test_index <- createDataPartition(y = movielens$rating, times = 1, p = 0.1, list = FALSE)
```

```r
edx <- movielens[-test_index,]
temp <- movielens[test_index,]

# Make sure userId and movieId in final hold-out test set are also in edx set
final_holdout_test <- temp %>%
  semi_join(edx, by = "movieId") %>%
  semi_join(edx, by = "userId")

# Add rows removed from final hold-out test set back into edx set
removed <- anti_join(temp, final_holdout_test)
```

```
## Joining with `by = join_by(userId, movieId, rating, timestamp, title, genres)`
```

```r
edx <- rbind(edx, removed)

rm(dl, ratings, movies, test_index, temp, movielens, removed)
```

We have now successfully downloaded all necessary packages, installed the datasets, and partitioned Movielens into our 'edx' training set and 'final_holdout_test' set.

Exploratory Data Analysis (EDA)

We will start by familiarizing ourselves with the dataset.

```r
# getting familiar with edx
head(edx)
```

```
##   userId movieId rating timestamp                         title
## 1      1     122      5 838985046              Boomerang (1992)
## 2      1     185      5 838983525               Net, The (1995)
## 3      1     231      5 838983392          Dumb & Dumber (1994)
## 4      1     292      5 838983421               Outbreak (1995)
## 5      1     316      5 838983392               Stargate (1994)
## 6      1     329      5 838983392 Star Trek: Generations (1994)
##                           genres
## 1                Comedy|Romance
## 2          Action|Crime|Thriller
## 3                         Comedy
## 4  Action|Drama|Sci-Fi|Thriller
## 5       Action|Adventure|Sci-Fi
## 6 Action|Adventure|Drama|Sci-Fi
```

```r
dim(edx)
```

```
## [1] 9000061       6
```

```r
names(edx)
```

```
## [1] "userId"    "movieId"   "rating"    "timestamp" "title"     "genres"
```

Here, we can see that 'edx' contains 9 million rows, or observations, and 6 columns:

userId: A unique ID given to each user

movieId: A unique ID given to each movie

title: The title of the movie

rating: A star-based rating from 0 to 5 for each movie

timestamp: The data and time for when the rating was given to the movie

genres: The genre of the movie

The 'rating' column is what we will be trying to predict for our model. That means the other 5 columns will be used as predictors. Using the following code, we can see that the two predictors 'movieId' and 'title' columns both predict the same movie.

```
edx %>% select(movieId, title) %>% head(10) # see that movieId and title represent the same movie
```

```
##    movieId                          title
## 1      122                Boomerang (1992)
## 2      185                 Net, The (1995)
## 3      231            Dumb & Dumber (1994)
## 4      292                 Outbreak (1995)
## 5      316                 Stargate (1994)
## 6      329 Star Trek: Generations (1994)
## 7      355         Flintstones, The (1994)
## 8      356             Forrest Gump (1994)
## 9      362         Jungle Book, The (1994)
## 10     364           Lion King, The (1994)
```

Thus, it is only necessary to use 'movieId'.

Additionally, we can see that 'final_holdout_test' has the same amount of columns but only 10% of the observations from the original dataset.

```
dim(final_holdout_test)
```

```
## [1] 999993      6
```

From now on, we will not use 'final_holdout_test' until we perform the final validation.

We will continue by analyzing how different columns, or predictors, affect how the ratings are distributed. This will assist us in making our model later.

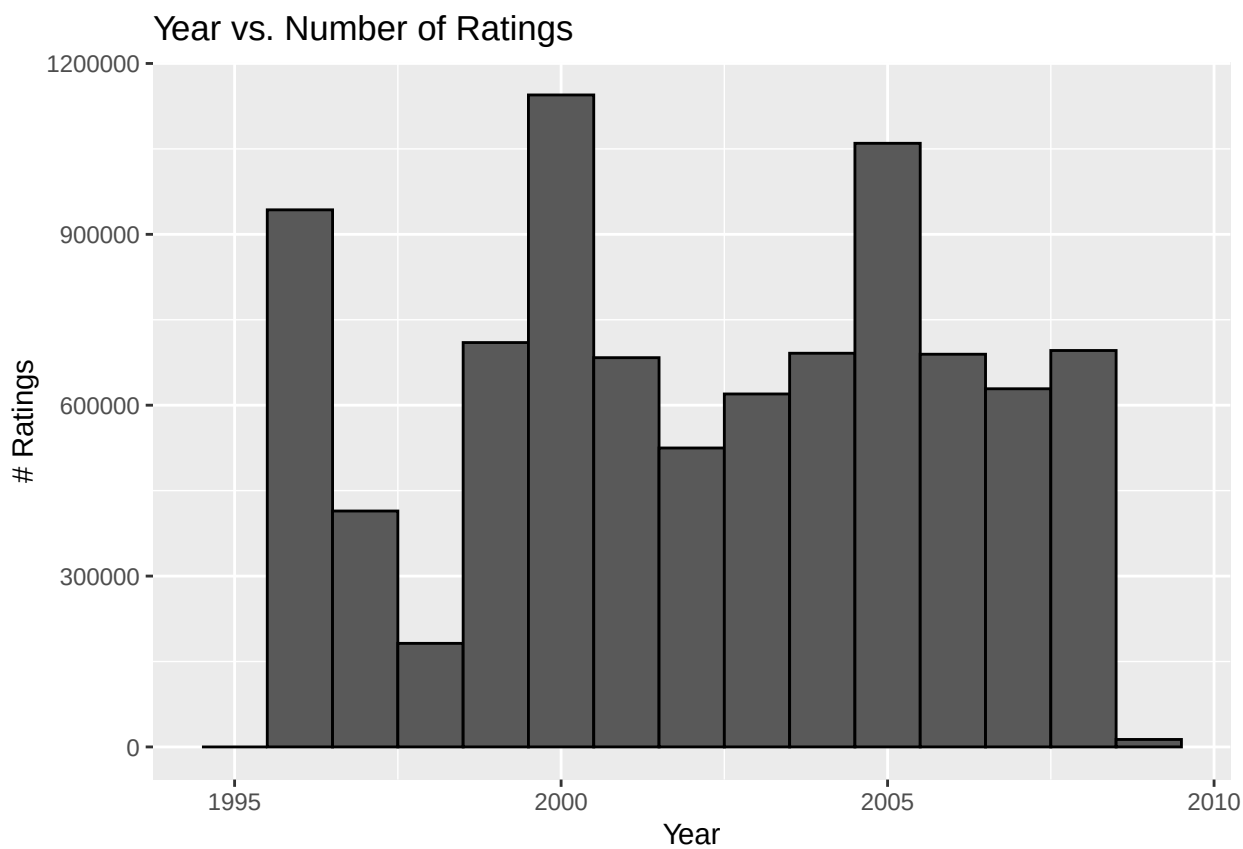
Timestamp and Year

Here, we simplify the timestamp predictor to just give us the year of the rating. We update the dataset to 'edx_new' which contains 'edx' and the new 'year' variable.

```
#analyzing the relationships between predictors and ratings
#simplifying the timestamp variable to year
#creating new edx data set with year from timestamp
edx_new <- edx %>%
  mutate(year=year(as_datetime(timestamp)))
```

Next, we create a histogram of the year and the amount of ratings it received.

```
edx_new %>%
  ggplot(aes(year)) +
  geom_histogram(binwidth = 1, color = "black") +
  ggtitle("Year vs. Number of Ratings") +
  xlab("Year") +
  ylab("# Ratings")
```



We see from here that the data fluctuates from high to low over the years and that the ratings are not normally distributed.

MovieID

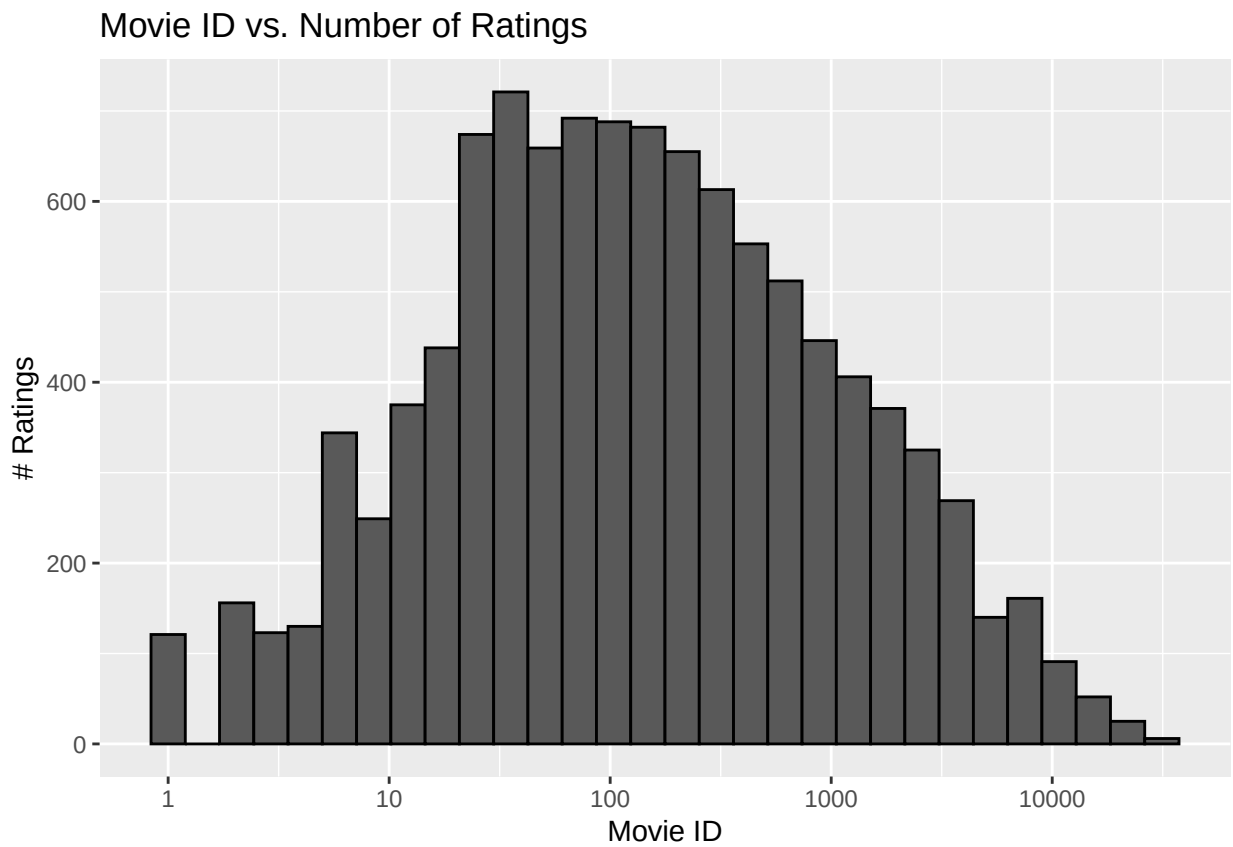
Next, this code will create a histogram of the Movie ID and the ratings.

```
#movieId and ratings
length(unique(edx_new$movieId)) # 10677 observations
```

```
## [1] 10677
```

```
edx_new %>%
  count(movieId) %>%
  ggplot(aes(n)) +
```

```
  geom_histogram(bins= 30, color = "black") +
  scale_x_log10() +
  ggtitle("Movie ID vs. Number of Ratings") +
  xlab("Movie ID") +
  ylab("# Ratings")
```



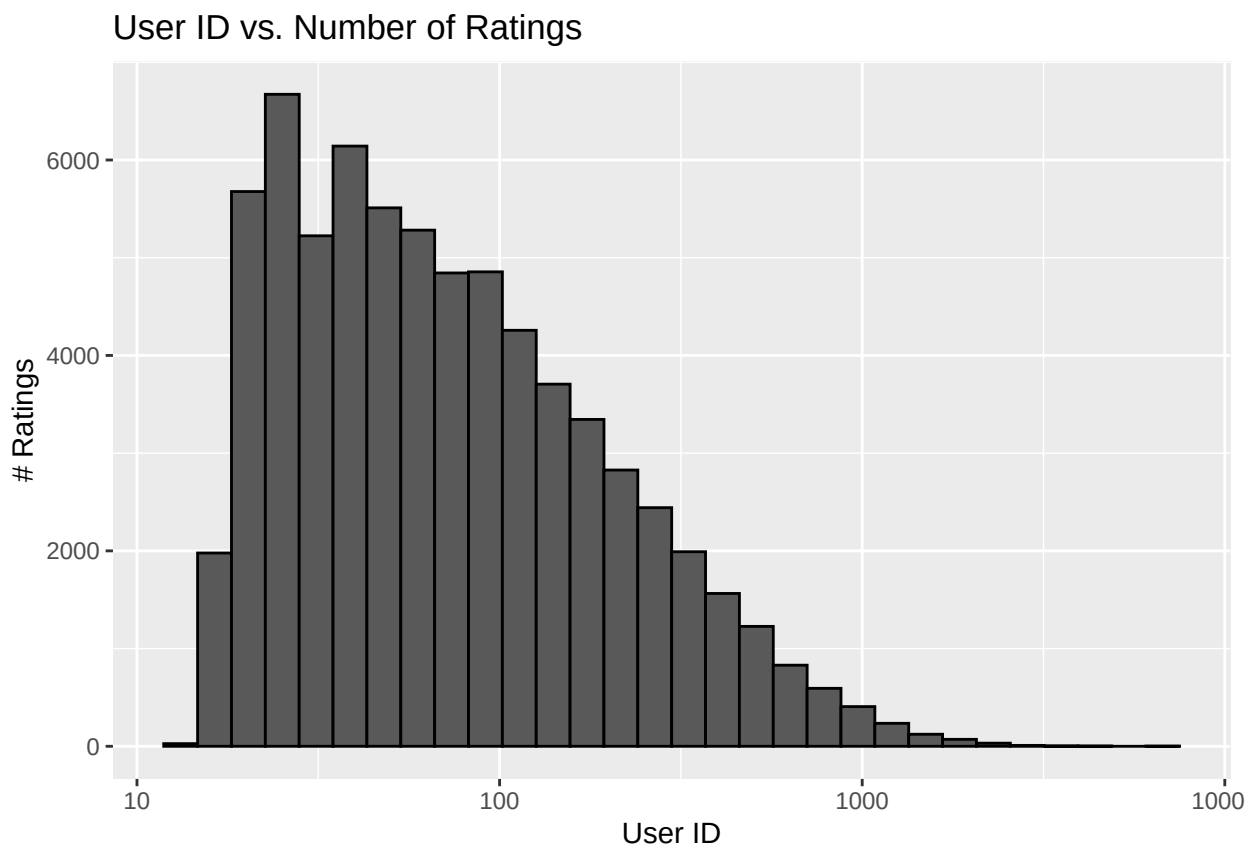
We can see from the histogram that some movies are rated more often than others, and that the ratings here are approximately normally distributed.

UserID

```
#userId and ratings
length(unique(edx_new$userId)) # 69878 observations
```

```
## [1] 69878
```

```
edx_new %>%
  count(userId) %>%
  ggplot(aes(n)) +
  geom_histogram(bins=30, color="black") +
  scale_x_log10() +
  ggtitle("User ID vs. Number of Ratings") +
  xlab("User ID") +
  ylab("# Ratings")
```



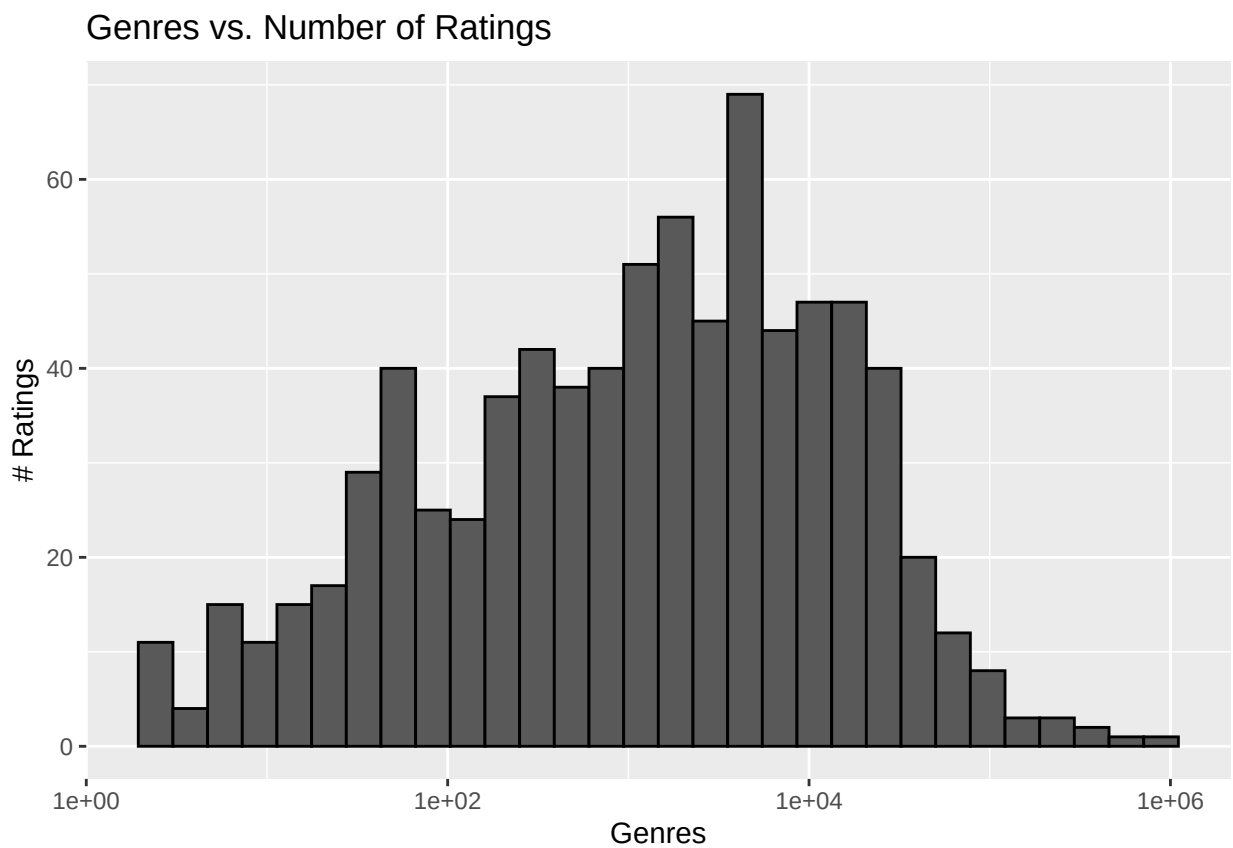
The histogram here looks approximately distributed, however towards the left side. It also shows that some users tend to rate more movies than others.

Genres

```
#genres and ratings
length(unique(edx_new$genres)) #797 observations
```

```
## [1] 797
```

```
edx_new %>% count(genres) %>%
  ggplot(aes(n)) +
  geom_histogram(bins=30, color= "black") +
  scale_x_log10() +
  ggtitle("Genres vs. Number of Ratings") +
  xlab("Genres") +
  ylab("# Ratings")
```



Here we can see from the histogram that some genres and combination of genres are rated more than others. The histogram has two peaks, so instead of being normally distributed, it is bimodally distributed.

Ratings

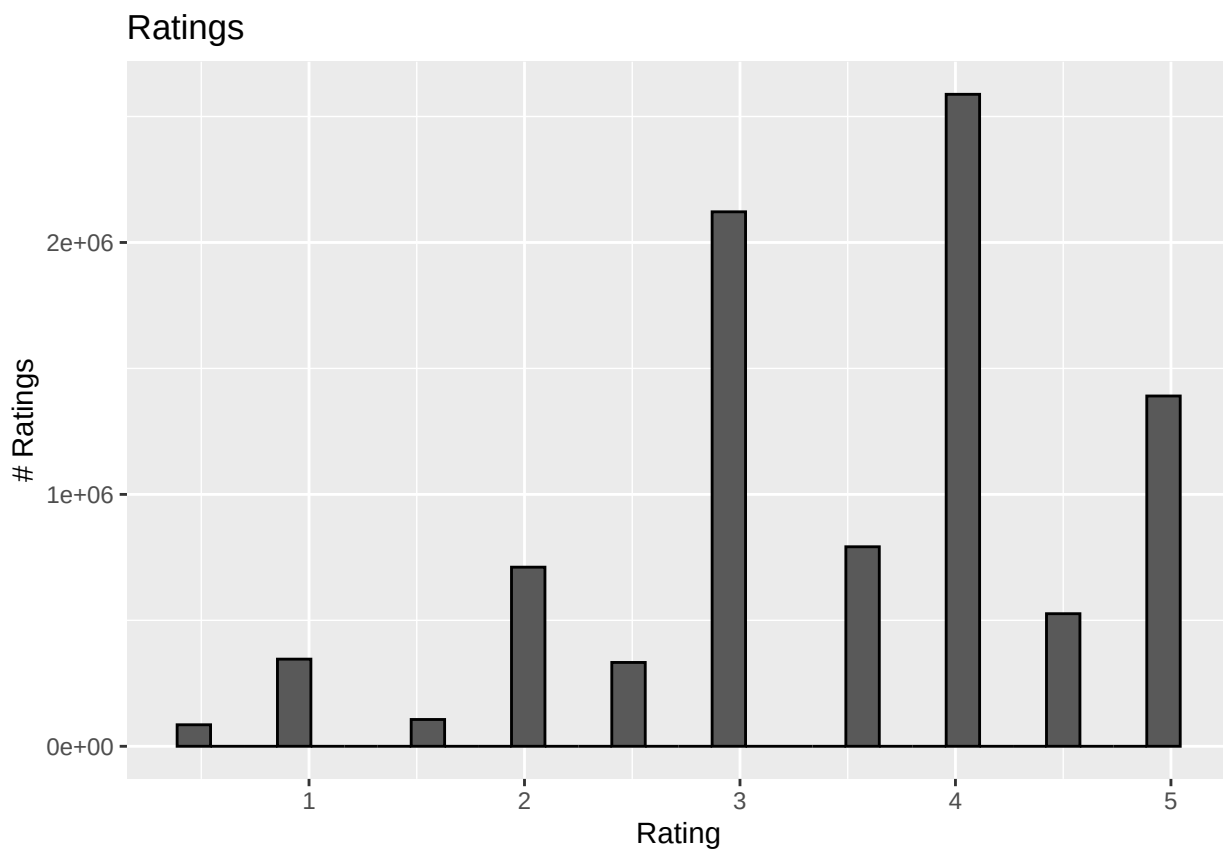
Although it might seem useless, plotting the ratings against the number of ratings can still help us with our analysis.

```
#distribution of ratings
length(unique(edx_new$rating)) #10 observations
```

```
## [1] 10
```

```
edx_new %>%
  ggplot(aes(rating)) +
  geom_histogram(color = "black") +
  ggtitle("Ratings") +
  xlab("Rating") +
  ylab("# Ratings")
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



From this code we can see that users tend to prefer to rate movies high than low. There is a much higher amount of 3's and 4's compared to 1's and 2's. The number of ratings also seems to dip whenever the graph shows a half star rating, showing users have a tendency to provide whole number ratings over decimal ratings.

Methods and Modeling Approach

Having analyzed our data and the different predictors, we can now begin creating our model. We will be using the 'edx_new' dataset instead of 'edx' since we have created a new variable 'year'. We will partition the 'edx_new' dataset into a 'training_set' and a 'test_set' which is different from the 'final_holdout_test' set. The 'training_set' will contain 80% of the data from 'edx_new' and the 'test_set' will contain 20% of 'edx_new'.

```
#Partition 20% of the data for the testing set
set.seed(1) #set the seed for reproducibility
test_index <- createDataPartition(edx_new$rating, times=1, p=0.2, list=FALSE)
test_set <- edx_new[test_index,]
training_set <- edx_new[-test_index,] #80% of the data used for the training set
```

Next, we will define the Random Mean Squared Error (RMSE) function, because that is what we will be using to measure the error between our predicted and actual values.

We can write the formula for RMSE in R with the following code:

```r
#defining the RMSE function
rmse <- function(actual, predicted){
  sqrt(mean((actual - predicted)^2))
}
```

As we have seen from our data analysis, the ratings of movies were caused by multiple predictors and the ratings were mostly normally distributed.

We will then be using a linear model for our prediction model, as it will also help us add in potential biases from the predictors and improve our RMSE.

The formula for a linear model is:

$Y_{u,i} = \mu + \epsilon_{u,i}$

We can begin by using this baseline linear model, where $\mu + \epsilon_{u,i}$ is the mean rating for all the movies. We can add potential biases later.

The average rating is found with this code:

```r
mu <- mean(training_set$rating) #represents the global average rating
mu
```

```
## [1] 3.51238
```

Now we can use the RMSE function to see the difference between our global average rating and the actual ratings.

```r
rmse(mu, test_set$rating)
```

```
## [1] 1.059648
```

This RMSE is way above what we want our RMSE, so we will have to use additional methods the achieve a smaller RMSE

Bias

Since our initial prediction's RMSE was too high, we will account for each predictor's bias in order to get a lower RMSE.

The formula for a linear model with added bias is:

$\hat{Y}_{u,i} = \mu + b_i + \epsilon_{i,u}$

And the formula for calculating predictor's bias is:

$\hat{b}_i = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{\mu})$

Therefore, we write those formulas in R with the following code:

```r
#creating a function to get the bias
movie_bias <- function(column_name){
  training_set %>%
    group_by(!!sym(column_name)) %>%
    summarize(bias=mean(rating - mu, na.rm = TRUE))
}
```

Now, we add the bias to our initial prediction.

```
#creating a prediction function now that we have the bias
movie_predicted <- function(movie_bias, column_name){
  test_set %>%
    left_join(movie_bias, by = column_name) %>%
    mutate(prediction = mu + replace_na(bias,0)) %>%
    pull(prediction)
}
```

Now we can calculate our RMSE after we've combined the bias with our initial prediction.

```
movie_rmse <- function(column_name){
  bias <- movie_bias(column_name)
  predicted <- movie_predicted(bias, column_name)
  rmse(predicted,test_set$rating)
}
```

We get the RMSE for each predictor bias with the following code:

```
movie_rmse('movieId') # approximately normally distributed so the RMSE is lower
```

```
## [1] 0.9431811
```

```
movie_rmse('userId') # approximately normally distributed so the RMSE is lower
```

```
## [1] 0.9781292
```

```
movie_rmse('genres') # not normally distributed so the RMSE is higher
```

```
## [1] 1.017305
```

```
movie_rmse('year') # not normally distributed so the RMSE is higher
```

```
## [1] 1.057902
```

From the resulting RMSE's we can see that the bias from movieId and userId significantly lowers the RMSE. This is because those predictors are approximately normal. Genres is not normally distributed, but still slightly improves the RMSE. The year variable did not reduce the RMSE by much.

To reduce the RMSE even further, we will combine all of our biases into the linear model with our initial prediction. The predictors, 'movieId', 'userId', and 'genres', will be used as they improved the RMSE. We will not be using the 'year' predictor because it did not improve the RMSE at all.

Our formula for the combined biases is the following:

$\hat{b}_u = \frac{1}{N}\sum_{i=1}^{N}\left(y_{u,i} - \hat{b}_i - \hat{\mu}\right)$

We translate that formula into code and combine the biases as shown here:

```r
#combining different biases for lower RMSE
#movieId, userId, and genres improved RMSE
movieId_bias <- movie_bias('movieId')
#combining movieId and userId bias
movie_user_bias <- training_set %>%
  left_join(movieId_bias, by = 'movieId') %>%
  group_by(userId) %>%
  summarize(user_bias=mean(rating-mu-bias, na.rm = TRUE))
#combining movieId, userId, and genre bias
all_bias <- training_set %>%
  left_join(movieId_bias, by="movieId") %>%
  left_join(movie_user_bias, by="userId") %>%
  group_by(genres) %>%
  summarize(genre_bias = mean(rating-mu-bias-user_bias,na.rm=TRUE))
```

While we're calculating biases we make sure to get rid of any NA values so that we do not receive an error.

Now that we have our combined biases we can include it into our linear model which can be given by the following equation:

$\hat{Y}_{u,i,g} = \mu + b_i + b_u + b_g + \epsilon_{i,u}$

Here we translate that formula into code:

```r
#adding the combined biases to our prediction
prediction <- test_set %>%
  left_join(movieId_bias, by = 'movieId') %>%
  left_join(movie_user_bias, by = 'userId') %>%
  left_join(all_bias, by = 'genres') %>%
  mutate(
    bias = replace_na(bias,0),
    user_bias = replace_na(user_bias,0),
    genre_bias = replace_na(genre_bias,0),
    prediction = mu+bias+user_bias+genre_bias) %>%
  pull(prediction)
```

Now we can calculate the RMSE.

```r
rmse(prediction,test_set$rating) #calculating the new RMSE with the combined biases
```

```
## [1] 0.8651779
```

```r
#RMSE is greatly improved with combined biases
```

We can see that our RMSE greatly improved after combining our biases, however, it is still not at our target RMSE.

Regularization

The final step that we will be taking to improving the RMSE of our linear prediction model is using Regularization. Regularization is a technique that we use in data analysis and machine learning that prevents overfitting and reduces the error of a model. Regularization means that we reduce errors by adding a penalty

(λ). λ reduces the errors of the model by preventing it from trying to fit outliers and other unnecessary parts in the training data.

Formula for Regularization:

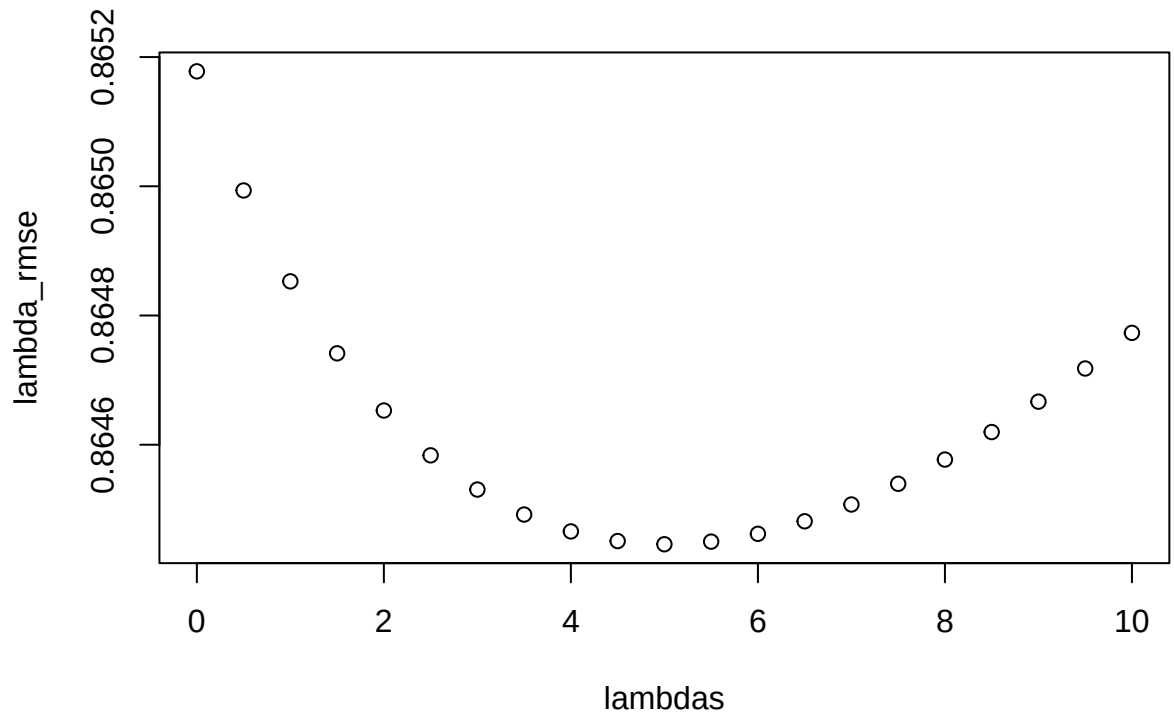
$\hat{b}_u = \frac{1}{n_u+\lambda} \sum_{i=1}^{n_u} (y_{u,i} - \hat{\mu})$

We begin Regularization by trying to find the best λ that will fit our code. We create a sequence of different λ and find the RMSE for each.

```
lambdas <- seq(0, 10, 0.5)
lambda_rmse <- sapply(lambdas,function(lambda){

  movieId_bias <- training_set %>%
    group_by(movieId) %>%
    summarize(bias = sum(rating-mu)/(n()+lambda))

  movie_user_bias <- training_set %>%
    left_join(movieId_bias, by = 'movieId') %>%
    group_by(userId) %>%
    summarize(user_bias=sum(rating - mu - bias)/(n()+lambda))

  all_bias <- training_set %>%
    left_join(movieId_bias, by="movieId") %>%
    left_join(movie_user_bias, by="userId") %>%
    group_by(genres) %>%
    summarize(genre_bias = sum(rating - mu - bias - user_bias)/(n()+lambda))

  prediction <- test_set %>%
    left_join(movieId_bias, by = 'movieId') %>%
    left_join(movie_user_bias, by = 'userId') %>%
    left_join(all_bias, by = 'genres') %>%
    mutate(
      bias = replace_na(bias,0),
      user_bias = replace_na(user_bias,0),
      genre_bias = replace_na(genre_bias,0),
      prediction = mu+bias+user_bias+genre_bias) %>%
    pull(prediction)
  rmse(prediction,test_set$rating)
})
```

We will plot the RMSE for each λ, and we can see the best λ for our model. The best λ will be the minimum of the plot.

```
plot(lambdas, lambda_rmse)
```



Since the best λ is the minimum we use the following code to find the best λ to fit our model.

```
#getting the best lambda that gets us our lowest RMSE
lambda <- lambdas[which.min(lambda_rmse)]
lambda # best lambda is 5
```

```
## [1] 5
```

Final Test and Result

After using the combined biases and using Regularization to improve our linear model, we are now ready to do the final test. In this final test, 'edx_new' will be our training set and 'final_holdout_test' will be our test set.

```
#beginning test on final holdout test set
final_mu <- mean(edx$rating) #now we grab the mu of the edx data set to test on the final holdout set
#combine the biases one more time
final_movieId_bias <- edx %>%
  group_by(movieId) %>%
  summarize(bias = sum(rating-final_mu)/(n()+lambda))

final_movie_user_bias <- edx %>%
  left_join(final_movieId_bias, by = 'movieId') %>%
  group_by(userId) %>%
  summarize(user_bias=sum(rating - final_mu - bias)/(n()+lambda))
```

```r
final_all_bias <- edx %>%
  left_join(final_movieId_bias, by="movieId") %>%
  left_join(final_movie_user_bias, by="userId") %>%
  group_by(genres) %>%
  summarize(genre_bias = sum(rating - final_mu - bias - user_bias)/(n()+lambda))

prediction <- final_holdout_test %>%
  left_join(final_movieId_bias, by = 'movieId') %>%
  left_join(final_movie_user_bias, by = 'userId') %>%
  left_join(final_all_bias, by = 'genres') %>%
  mutate(
    bias=replace_na(bias, 0),
    user_bias = replace_na(user_bias,0),
    genre_bias = replace_na(genre_bias,0),
    prediction = final_mu+bias+user_bias+genre_bias) %>%
  pull(prediction)
```

Now we calculate the RMSE.

```r
rmse(prediction,final_holdout_test$rating) #Final RMSE is 0.8646798
```

```
## [1] 0.8646798
```

We are below our target RMSE.

Conclusion

Our final RMSE was 0.8646798. This means that we are below the target RMSE of 0.8649.

After downloading the dataset and partitioning it into 'edx' and 'final_holdout_test' we performed our exploratory data analysis and saw that the the predictors were mostly normally distributed. Using that we justified using a linear model using the Central Limit Theorem. Calculating the RMSE of our initial prediction, we used combined biases and regularization to to improve the prediction and used Regularization to improve our model.

A limitation that we faced was that we were unable to use all of the predictors within our model. This was because some of the predictors when plotted against the movie ratings, were not normally distributed. This limited us from using them in our linear model which relies on the predictors being normally distributed.

In the future, all the predictors will be used and different models as well. This will produce an even more accurate prediction model. In addition, more advanced models such as Random Forest models, and K-nearest neighbors models will be used for even more accurate predictions.

References

https://rafalab.dfci.harvard.edu/dsbook

https://learning.edx.org/course/course-v1:HarvardX+PH125.7x+1T2025/home

https://learning.edx.org/course/course-v1:HarvardX+PH125.8x+1T2025/home

https://learning.edx.org/course/course-v1:HarvardX+PH125.9x+1T2025/home